Diseño Web 1
ADSO-CTMA

Servicio Nacional de Aprendizaje SENA

Deimer Miranda Montoya, MSc.(c).
damiranda@sena.edu.co

Normas del curso



Cumple con los horarios y demuestra compromiso académico en cada sesión.



Mantén una actitud profesional, ética y orientada al aprendizaje continuo.



Comunica tus ideas con claridad, respeto y fundamento técnico.



Participa activamente y contribuye al trabajo colaborativo.



Haz uso responsable de los recursos, equipos e instalaciones.

Reflexión inicial

”La única persona que puede decirte ‘no puedes lograrlo’ eres tú misma, y no debes escucharla”
Jessica Ennis.



Flexbox

Distribuir y alinear elemen-
tos dentro de un contenedor

Propósito de la clase

Al finalizar la clase, el aprendiz estará en capacidad de:

- Explicar qué es Flexbox y para qué se utiliza.
- Diferenciar contenedor flex y elementos flex.
- Comprender el eje principal y el eje transversal.
- Aplicar `display: flex`.
- Utilizar `flex-direction`.
- Distribuir elementos con `justify-content`.
- Alinear elementos con `align-items`.
- Separar elementos mediante gap.

Pregunta orientadora

¿Cómo podemos organizar varios elementos sin moverlos manualmente uno por uno?

¿Por qué necesitamos Flexbox?

Hasta ahora hemos trabajado elementos individuales:

- Tamaño de una caja.
- Espacio interno y externo.
- Bordes.
- Posicionamiento puntual.

Pero muchas interfaces necesitan organizar varios elementos:

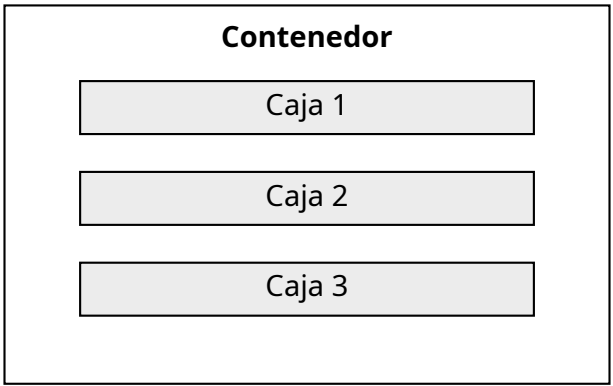
- Menús.
- Tarjetas.
- Botones.
- Galerías.
- Secciones de contenido.

Idea clave

Flexbox permite distribuir elementos dentro de un contenedor de forma más flexible y controlada.

Problema inicial

Supongamos tres cajas dentro de un contenedor.
Sin Flexbox, normalmente aparecen una debajo de otra si son elementos de bloque.



HTML base del ejemplo

Para los ejemplos usaremos una estructura pequeña:

```
<div class="contenedor">
<div class="caja">Caja 1</div>
<div class="caja">Caja 2</div>
<div class="caja">Caja 3</div>
</div>
```

Observación

El elemento con clase `contenedor` agrupa las tres cajas.

CSS base del ejemplo

Antes de usar Flexbox, aplicamos estilos simples:

```css
.contenedor {
    border: 2px solid black;
    padding: 10px;
}

.caja {
    border: 1px solid gray;
    padding: 15px;
    margin: 5px;
}
```

Importante

Aquí todavía no hay Flexbox. Solo estamos haciendo visibles las cajas.

Fundamentos de Flexbox

Contenedor flex y elementos flex

¿Qué es Flexbox?

Flexbox es un modelo de diseño de CSS pensado para organizar elementos dentro de un contenedor.
Su nombre proviene de:

Flexible Box Layout

En español:

Modelo de cajas flexibles

Idea clave

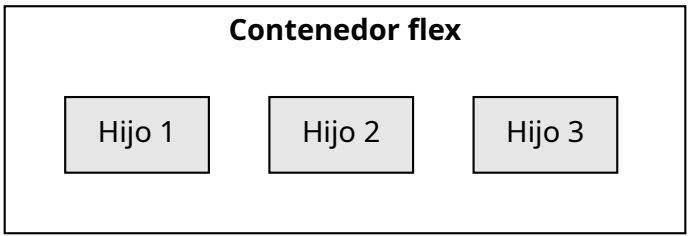
Flexbox no se aplica a una caja aislada, sino a la relación entre un contenedor y sus elementos internos.

Contenedor flex y elementos flex

Flexbox trabaja con dos conceptos principales:

Contenedor flex: elemento padre al que se le aplica `display: flex`.

Elementos flex: elementos hijos directos del contenedor flex.



Activar Flexbox

Flexbox se activa en el contenedor:

```
.contenedor {
    display: flex;
}
```

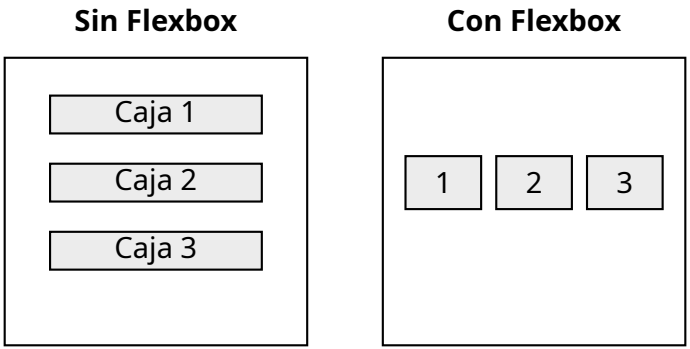
Efecto

Los hijos directos de `.contenedor` pasan a comportarse como elementos flex.

Importante

`display: flex` se escribe en el padre, no en cada hijo.

Antes y después de display flex



Ejemplo mínimo completo

HTML:

```
<div class="contenedor">
<div class="caja">1</div>
<div class="caja">2</div>
<div class="caja">3</div>
</div>
```

CSS:

```
.contenedor {
    display: flex;
}
```

Resultado

Las cajas hijas se organizan inicialmente en una fila.

Ejes de Flexbox

Eje principal y eje transversal

¿Por qué hablar de ejes?

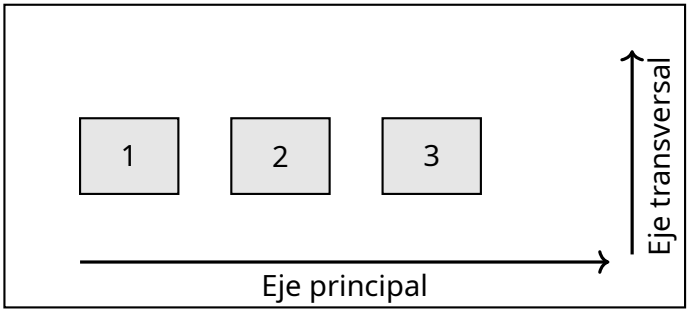
Flexbox organiza los elementos siguiendo dos ejes:

- Eje principal.
- Eje transversal.

Idea clave

Muchas propiedades de Flexbox no significan “horizontal” o “vertical” de forma fija. Dependen del eje principal definido.

Eje principal y eje transversal



Caso inicial

Por defecto, Flexbox organiza los elementos en dirección horizontal.

Eje principal

El eje principal indica la dirección en la que se distribuyen los elementos flex.
Por defecto:

$$\text{izquierda} \rightarrow \text{derecha}$$

En ese caso, los elementos se organizan en una fila.

Propiedad relacionada

La propiedad que controla el eje principal es:
`flex-direction`

Eje transversal

El eje transversal cruza al eje principal.
Si el eje principal es horizontal, el eje transversal será vertical.

eje principal: horizontal

eje transversal: vertical

Idea

Algunas propiedades alinean sobre el eje principal y otras sobre el eje transversal.

flex-direction

Definir la dirección del eje principal

¿Qué hace flex-direction?

La propiedad `flex-direction` define la dirección del eje principal.
Valores principales:

- `row`
- `row-reverse`
- `column`
- `column-reverse`

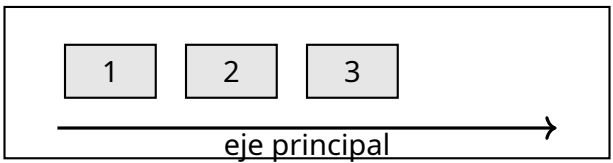
Sintaxis

```
flex-direction: valor;
```

flex-direction: row

Es el valor predeterminado.

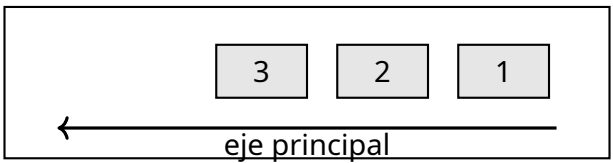
```
.contenedor {
    display: flex;
    flex-direction: row;
}
```



flex-direction: row-reverse

Invierte la dirección horizontal.

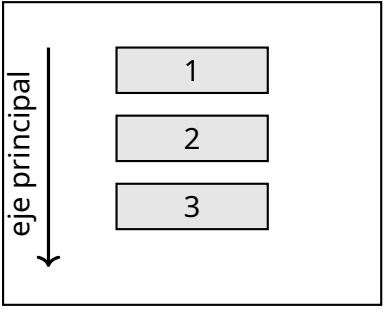
```
.contenedor {
    display: flex;
    flex-direction: row-reverse;
}
```



flex-direction: column

Cambia el eje principal a vertical.

```
.contenedor {
    display: flex;
    flex-direction: column;
}
```



flex-direction: column-reverse

Invierte la dirección vertical.

```
.contenedor {
    display: flex;
    flex-direction: column-reverse;
}
```

Resultado

El eje principal es vertical, pero el orden visual se presenta desde abajo hacia arriba.

Resumen de flex-direction

Valor	Comportamiento
`row`	organiza de izquierda a derecha.
`row-reverse`	organiza de derecha a izquierda.
`column`	organiza de arriba hacia abajo.
`column-reverse`	organiza de abajo hacia arriba.

Importante

Cambiar `flex-direction` también cambia la interpretación de otras propiedades.

justify-content

Distribuir elementos sobre el eje principal

¿Qué hace justify-content?

La propiedad `justify-content` controla la distribución de los elementos sobre el **eje principal**.

Idea clave

Para interpretar correctamente `justify-content`, primero debemos identificar cuál es el eje principal.

Por defecto:

```
flex-direction: row;
```

Por lo tanto, el eje principal es horizontal.

HTML de referencia

Continuaremos utilizando:

```
<div class="contenedor">
<div class="caja">1</div>
<div class="caja">2</div>
<div class="caja">3</div>
</div>
```

CSS inicial:

```
.contenedor {
    display: flex;
}
```

Solo modificaremos una propiedad en cada ejemplo.

Valores principales de justify-content

En esta clase trabajaremos con:

- `flex-start`
- `center`
- `flex-end`
- `space-between`
- `space-around`
- `space-evenly`

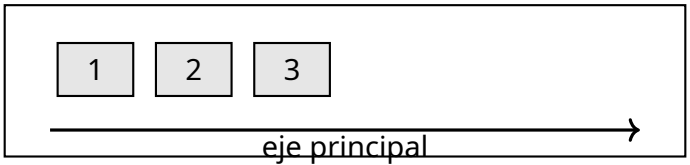
Sintaxis

```
justify-content: valor;
```

justify-content: flex-start

Es el comportamiento inicial.

```
.contenedor {
    display: flex;
    justify-content: flex-start;
}
```

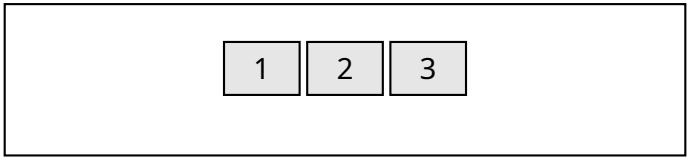


Los elementos se ubican al inicio del eje principal.

justify-content: center

Cambiamos solamente el valor:

```
.contenedor {
    display: flex;
    justify-content: center;
}
```



Resultado

El grupo de elementos se ubica en el centro del eje principal.

justify-content: flex-end

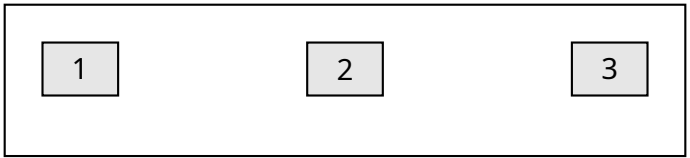
```css
.contenedor {
    display: flex;
    justify-content: flex-end;
}
```



Los elementos se ubican al final del eje principal.

justify-content: space-between

```css
.contenedor {
    display: flex;
    justify-content: space-between;
}
```

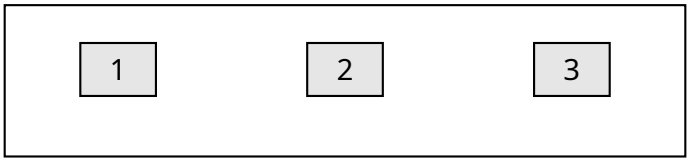


Comportamiento

El primer elemento queda al inicio, el último al final y el espacio restante se distribuye entre ellos.

justify-content: space-around

```
.contenedor {
    display: flex;
    justify-content: space-around;
}
```



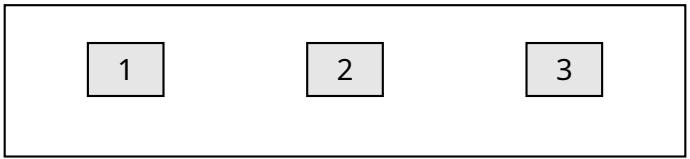
Cada elemento recibe espacio alrededor.

Observe

El espacio de los extremos suele ser menor que el espacio existente entre elementos.

justify-content: space-evenly

```css
.contenedor {
    display: flex;
    justify-content: space-evenly;
}
```



Comportamiento

Los espacios entre elementos y en los extremos son uniformes.

Comparación de justify-content

Valor	Resultado
`flex-start`	Elementos al inicio del eje.
`center`	Elementos agrupados en el centro.
`flex-end`	Elementos al final del eje.
`space-between`	Espacio solamente entre elementos.
`space-around`	Espacio alrededor de cada elemento.
`space-evenly`	Espacios uniformes.

¿Qué ocurre con column?

Cambiemos:

```
.contenedor {
    display: flex;
    flex-direction: column;
}
```

Ahora el eje principal es:

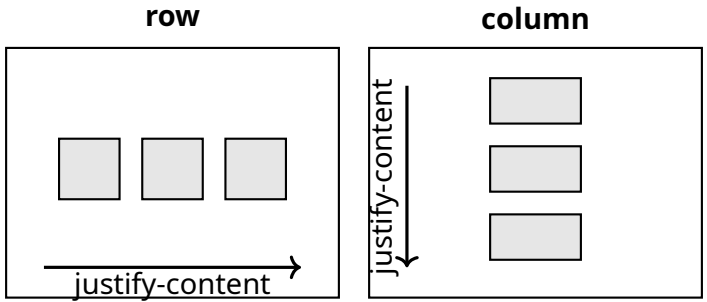
vertical

Por lo tanto:

`justify-content`

controlará la distribución vertical.

justify-content depende del eje



No memorice “horizontal”

Recuerde: `justify-content` trabaja sobre el **eje principal**.

align-items

Alinear elementos sobre el eje transversal

¿Qué hace align-items?

La propiedad `align-items` controla la alineación de los elementos sobre el **eje transversal**.
Si tenemos:

```
flex-direction: row;
```

entonces:

- eje principal: horizontal;
- eje transversal: vertical.

Sintaxis

```
align-items: valor;
```

Valores principales de align-items

Trabajaremos inicialmente con:

- `stretch`
- `flex-start`
- `center`
- `flex-end`

También existe:

`baseline`

que veremos brevemente.

Necesitamos altura para observarlo

Para visualizar mejor `align-items`, damos altura al contenedor:

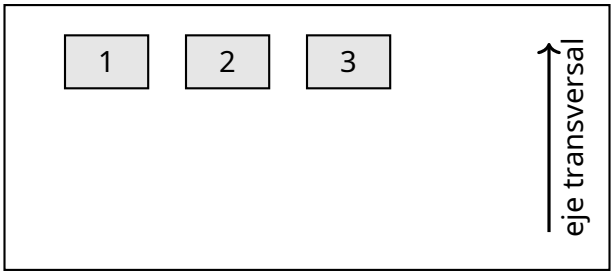
```
.contenedor {
    display: flex;
    height: 250px;
    border: 2px solid black;
}
```

¿Por qué?

Necesitamos espacio disponible sobre el eje transversal para observar cómo cambia la alineación.

align-items: flex-start

```
.contenedor {
    display: flex;
    height: 250px;
    align-items: flex-start;
}
```

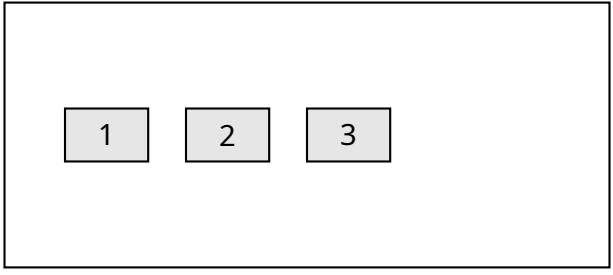


Los elementos se ubican al inicio del eje transversal.

align-items: center

Cambiamos solamente:

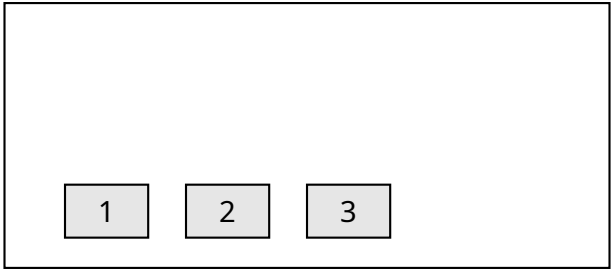
```
.contenedor {
    display: flex;
    height: 250px;
    align-items: center;
}
```



Los elementos se centran sobre el eje transversal.

align-items: flex-end

```css
.contenedor {
    display: flex;
    height: 250px;
    align-items: flex-end;
}
```



Los elementos se ubican al final del eje transversal.

align-items: stretch

`stretch` es el valor inicial.

```
.contenedor {
    display: flex;
    height: 250px;
    align-items: stretch;
}
```

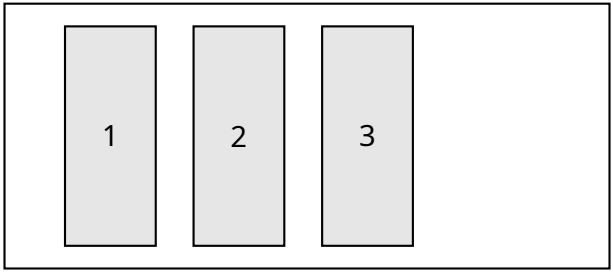
Si los elementos no tienen una dimensión definida sobre el eje transversal, pueden estirarse para ocupar el espacio disponible.

Importante

Si el elemento ya tiene una altura definida, el efecto puede ser diferente.



Representación de stretch



Idea

`stretch` aprovecha el espacio disponible sobre el eje transversal.

align-items: baseline

Otro valor disponible es:

```
.contenedor {
    display: flex;
    align-items: baseline;
}
```

Busca alinear los elementos teniendo en cuenta la línea base del contenido textual.

Uso

Puede resultar útil cuando los elementos contienen texto de diferentes tamaños.

justify-content y align-items

No confundir los dos ejes

La diferencia fundamental

`justify-content`

↓

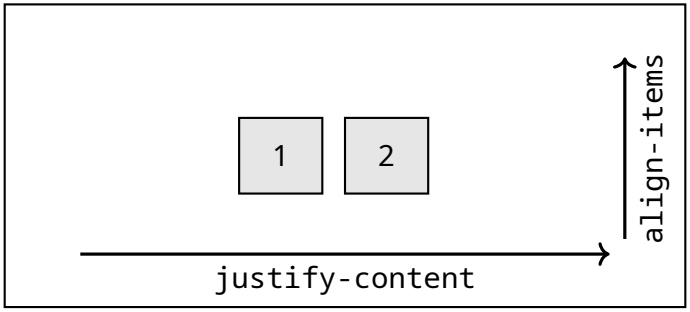
eje principal

`align-items`

↓

eje transversal

Mapa visual de los ejes



Caso mostrado:

```
flex-direction: row;
```

Centrar horizontal y verticalmente

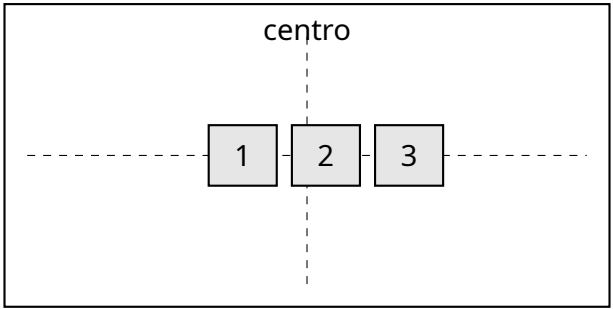
Ya podemos comprender este código:

```
.contenedor {
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Con `flex-direction: row`:

- `justify-content: center` centra sobre el eje horizontal.
- `align-items: center` centra sobre el eje vertical.

Centrado en ambos ejes



¿Y si usamos column?

Ahora:

```
.contenedor {
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
}
```

Cambio importante

El eje principal ahora es vertical.
Por tanto, `justify-content` centra verticalmente.

La regla que debemos recordar

No memorice:

```
justify = horizontal
  align = vertical
```

Eso solamente funciona cuando:

```
flex-direction: row
```

Recuerde mejor:

justify $\rightarrow$ eje principal
align $\rightarrow$ eje transversal

gap

Separar elementos flex

¿Qué problema resuelve gap?

Cuando varios elementos están juntos, normalmente necesitamos separación entre ellos.
Podríamos utilizar `margin`, pero Flexbox proporciona una propiedad específica:

gap

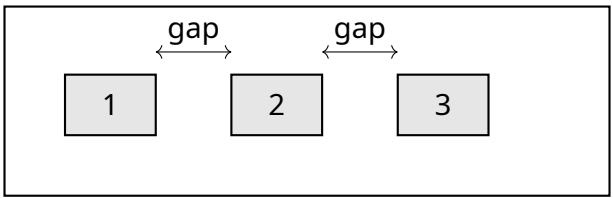
Función

gap define el espacio entre los elementos flex.

Sintaxis de gap

Ejemplo:

```
.contenedor {
    display: flex;
    gap: 20px;
}
```



Gap frente a margin

	gap	**margin**
Función	Separar elementos	Espacio exterior
Se aplica normalmente en	Contenedor	Elemento
Afecta extremos automáticamente	No	Puede hacerlo
Uso en Flexbox	Muy conveniente	También posible

Modificación progresiva

Antes:

```
.contenedor {
    display: flex;
}
```

Después:

```
.contenedor {
    display: flex;
    gap: 20px;
}
```

Observe

No fue necesario modificar cada elemento `.caja`.



flex-wrap

Permitir que los elementos cambien de línea

¿Qué ocurre cuando no hay espacio?

Por defecto, Flexbox intenta mantener todos los elementos dentro de una sola línea. Esto puede provocar que:

- Los elementos se reduzcan.
- El contenido quede muy comprimido.
- La distribución no sea conveniente.

Solución

Podemos permitir varias líneas mediante `flex-wrap`.

Valores principales de flex-wrap

Los valores más importantes son:

- `nowrap`
- `wrap`
- `wrap-reverse`

Valor predeterminado

```
flex-wrap: nowrap;
```

flex-wrap: nowrap

Ejemplo:

```
.contenedor {
    display: flex;
    flex-wrap: nowrap;
}

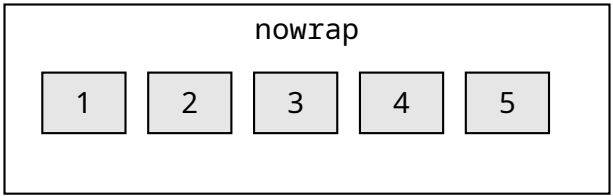
.caja {
    width: 200px;
}
```

Flexbox intenta conservar los elementos en una única línea.

Importante

Los elementos flex pueden reducirse si no existe espacio suficiente.

Representación de nowrap



flex-wrap: wrap

Modificamos:

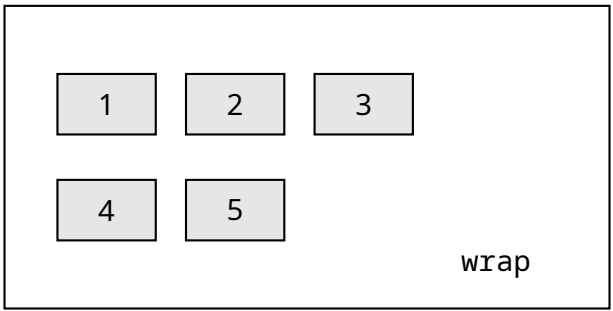
```
.contenedor {
    display: flex;
    flex-wrap: wrap;
}

.caja {
    width: 200px;
}
```

Resultado

Cuando los elementos dejan de caber, pueden continuar en una nueva línea.

Representación de wrap



Los elementos continúan en otra línea cuando no existe espacio suficiente.

flex-wrap: wrap-reverse

También existe:

```
.contenedor {
    display: flex;
    flex-wrap: wrap-reverse;
}
```

Permite varias líneas, pero invierte la dirección en la que se generan esas líneas.

Uso inicial

Es menos frecuente que `wrap`, pero forma parte de los valores disponibles.

Combinar gap y wrap

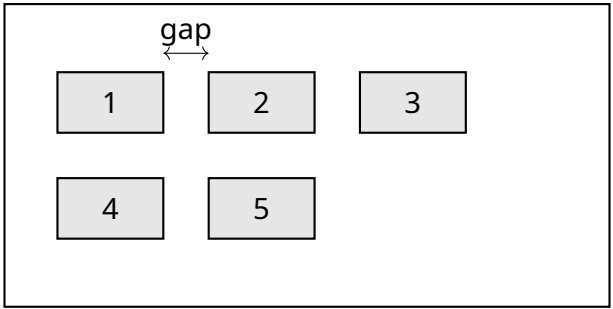
Ahora podemos combinar dos conceptos ya estudiados:

```
.contenedor {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}
```

Resultado

Los elementos pueden cambiar de línea y mantienen una separación uniforme.

Representación de gap + wrap



Ejemplo progresivo

Construir un pequeño menú con Flexbox

Paso 1: HTML

Utilicemos un menú pequeño:

```
<nav class="menu">
<a href="#">Inicio</a>
<a href="#">Servicios</a>
<a href="#">Contacto</a>
</nav>
```

Estructura

nav será el contenedor.
Los enlaces serán sus elementos hijos.

Paso 2: activar Flexbox

Primero:

```
.menu {
    display: flex;
}
```

Los enlaces pasan a ser elementos flex.

Paso 3: separar los enlaces

Agregamos:

```
.menu {
    display: flex;
    gap: 20px;
}
```

Ahora existe espacio uniforme entre los enlaces.

Paso 4: centrar el menú

Agregamos:

```css
.menu {
    display: flex;
    gap: 20px;
    justify-content: center;
}
```

Pregunta

¿Sobre qué eje se está realizando el centrado?

Paso 5: altura del menú

Ahora:

```
.menu {
    display: flex;
    gap: 20px;
    justify-content: center;
    height: 80px;
}
```

Todavía los elementos no están necesariamente centrados sobre el eje transversal.

Paso 6: centrar en ambos ejes

Finalmente:

```
.menu {
    display: flex;
    gap: 20px;
    justify-content: center;
    align-items: center;
    height: 80px;
}
```

Resultado

Los enlaces quedan centrados sobre ambos ejes.

Análisis del menú

En:

```
flex-direction: row
```

tenemos:

`display: flex:` activa Flexbox.

`gap:` separa los enlaces.

`justify-content:` distribuye horizontalmente.

`align-items:` alinea verticalmente.

Aplicación

Distribuir tarjetas sencillas

HTML inicial

Utilice:

```
<section class="tarjetas">

<article class="tarjeta">
<h2>Servicio 1</h2>
</article>

<article class="tarjeta">
<h2>Servicio 2</h2>
</article>

<article class="tarjeta">
<h2>Servicio 3</h2>
</article>

</section>
```




Paso 1: hacer visibles las tarjetas

```css
.tarjeta {
    width: 200px;
    padding: 20px;
    border: 1px solid gray;
}
```

Observe primero el comportamiento antes de aplicar Flexbox.

Paso 2: activar Flexbox

```css
.tarjetas {
    display: flex;
}
```

Observe

¿Qué cambió respecto al flujo anterior?

Paso 3: agregar separación

```css
.tarjetas {
    display: flex;
    gap: 20px;
}
```

Paso 4: permitir varias líneas

```
.tarjetas {
    display: flex;
    gap: 20px;
    flex-wrap: wrap;
}
```

Reduzca el ancho de la ventana y observe qué ocurre.

Paso 5: centrar las tarjetas

Agregue:

```
.tarjetas {
    display: flex;
    gap: 20px;
    flex-wrap: wrap;
    justify-content: center;
}
```

Pregunta

¿Por qué utilizamos `justify-content` y no `align-items`?

Errores frecuentes

Comprender antes de combinar propiedades

Aplicar justify-content al hijo

Incorrecto:

```
.caja {
    justify-content: center;
}
```

si `.caja` no es un contenedor flex.
Correcto:

```
.contenedor {
    display: flex;
    justify-content: center;
}
```

Recuerde

Estas propiedades controlan la distribución de los hijos desde el contenedor.

Confundir justify-content y align-items

Error frecuente:

"justify siempre es horizontal"

Esto es falso.
La interpretación depende de:

`flex-direction`

Regla correcta

justify $\rightarrow$ eje principal
align $\rightarrow$ eje transversal

Usar margin para cada separación

Esto funciona:

```
.caja {
    margin-right: 20px;
}
```

Pero cuando el objetivo es simplemente separar elementos flex, suele ser más claro:

```
.contenedor {
    display: flex;
    gap: 20px;
}
```

Esperar wrap sin activarlo

Con:

```
.contenedor {
    display: flex;
}
```

el valor inicial es:

```
flex-wrap: nowrap;
```

Si queremos varias líneas:

```
.contenedor {
    display: flex;
    flex-wrap: wrap;
}
```

Verificación del Bloque 2

El aprendiz debe poder explicar:

- Qué eje controla `justify-content`.
- Qué eje controla `align-items`.
- Por qué los ejes dependen de `flex-direction`.
- Diferencia entre `space-between`, `space-around` y `space-evenly`.
- Qué función cumple gap.
- Qué diferencia existe entre nowrap y `wrap`.
- Por qué `display: flex` se aplica al contenedor.

align-content

Distribuir varias líneas dentro del contenedor

¿Qué hace align-content?

La propiedad `align-content` distribuye las **líneas flex** sobre el eje transversal.

Importante

Esta propiedad tiene sentido cuando:

- existe más de una línea;
- está activo `flex-wrap`;
- existe espacio disponible en el contenedor.

Utilizaremos más elementos para provocar varias líneas:

```
<div class=”contenedor”>
    <div class=”caja”>1</div>
    <div class=”caja”>2</div>
    <div class=”caja”>3</div>
    <div class=”caja”>4</div>
    <div class=”caja”>5</div>
    <div class=”caja”>6</div>
</div>
```

```css
.contenedor {
    display: flex;
    flex-wrap: wrap;
    height: 300px;
}

.caja {
    width: 180px;
}
```

Ahora pueden existir varias líneas dentro de un contenedor con altura disponible.

align-items frente a align-content

Estas propiedades no hacen lo mismo:

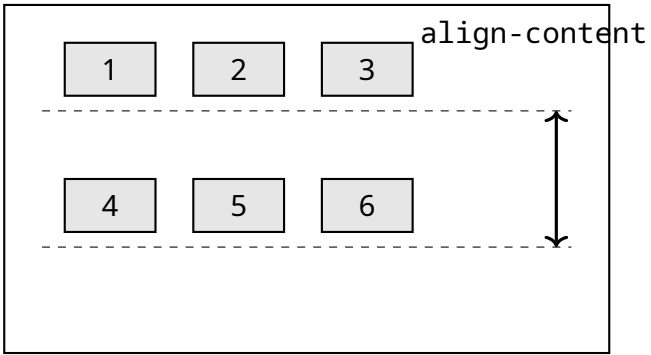
`align-items:` alinea los elementos dentro de una línea flex.

`align-content:` distribuye el conjunto de líneas flex cuando existen varias.

Regla práctica

Una línea $\rightarrow$ piense en `align-items`.
Varias líneas $\rightarrow$ puede intervenir `align-content`.

Representación conceptual



Valores principales de align-content

Podemos utilizar:

- `flex-start`
- `center`
- `flex-end`
- `space-between`
- `space-around`
- `space-evenly`
- `stretch`

align-content: flex-start

```
.contenedor {
    display: flex;
    flex-wrap: wrap;
    height: 300px;
    align-content: flex-start;
}
```

Las líneas se agrupan al inicio del eje transversal.

align-content: center

```
.contenedor {
    display: flex;
    flex-wrap: wrap;
    height: 300px;
    align-content: center;
}
```

Resultado

Las líneas flex se agrupan en el centro del eje transversal.

align-content: space-between

```
.contenedor {
    display: flex;
    flex-wrap: wrap;
    height: 300px;
    align-content: space-between;
}
```

La primera línea se ubica al inicio, la última al final y el espacio restante queda entre las líneas.

align-items y align-content

	align-items	**align-content**
Actúa sobre	Elementos	Líneas
Requiere wrap	No	Normalmente sí
Una sola línea	Sí tiene efecto	No suele tener efecto visible
Eje	Transversal	Transversal

Propiedades de los elementos flex

Controlar el comportamiento de cada hijo

Propiedades del contenedor y de los hijos

Hasta ahora hemos aplicado propiedades principalmente al contenedor:

- `display`
- `flex-direction`
- `justify-content`
- `align-items`
- `gap`
- `flex-wrap`
- `align-content`

Ahora estudiaremos propiedades que se aplican a los **elementos flex**.

Propiedades que estudiaremos

Sobre los elementos flex podemos utilizar:

- `flex-grow`
- `flex-shrink`
- `flex-basis`
- `flex`
- `order`
- `align-self`

Diferencia

Estas propiedades se escriben normalmente en los elementos hijos, no en el contenedor.

flex-grow

Distribuir el espacio disponible

¿Qué hace flex-grow?

`flex-grow` indica cuánto puede crecer un elemento flex cuando existe espacio disponible sobre el eje principal.

Valor inicial

`flex-grow: 0;`
Los elementos no crecen automáticamente para ocupar el espacio restante.

Ejemplo inicial

HTML:

```
<div class="contenedor">
    <div class="caja">1</div>
    <div class="caja">2</div>
    <div class="caja">3</div>
</div>
```

CSS:

```
.contenedor {
    display: flex;
}

.caja {
    width: 100px;
}
```

Si sobra espacio, este permanece libre.

Aplicar flex-grow

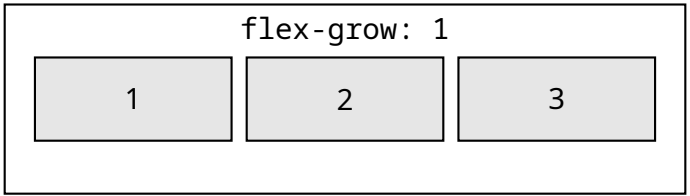
Modificamos los hijos:

```
.caja {
    flex-grow: 1;
}
```

Resultado

Los tres elementos participan de manera equivalente en la distribución del espacio disponible.

Representación de flex-grow: 1



Un elemento puede crecer más

Podemos modificar solamente uno:

```
.caja {
    flex-grow: 1;
}

.destacada {
    flex-grow: 2;
}
```

HTML:

```
<div class="caja">1</div>
<div class="caja destacada">2</div>
<div class="caja">3</div>
```

La caja 2 recibe una mayor proporción del espacio libre.

Interpretación de flex-grow

Los valores representan proporciones del **espacio libre**, no necesariamente el ancho total de los elementos.
Por ejemplo:

1 2 1

significa que el espacio disponible se distribuye proporcionalmente en esas partes.

Importante

`flex-grow: 2` no significa simplemente “el elemento será el doble de ancho”.

flex-shrink

Controlar la reducción de los elementos

¿Qué hace flex-shrink?

`flex-shrink` indica cuánto puede reducirse un elemento cuando no existe suficiente espacio.

Valor inicial

Normalmente:

```
flex-shrink: 1;
```

Por eso los elementos flex pueden reducirse cuando el contenedor es pequeño.

Ejemplo

Supongamos:

```
.contenedor {
    display: flex;
    width: 500px;
}

.caja {
    width: 250px;
}
```

Tres cajas necesitarían inicialmente:

$$250 + 250 + 250 = 750\text{px}$$

pero el contenedor tiene solamente 500px.

Evitar que un elemento se reduzca

Podemos indicar:

```
.importante {
    flex-shrink: 0;
}
```

Resultado

Ese elemento intenta conservar su tamaño base mientras otros elementos pueden absorber una mayor reducción.

grow frente a shrink

	flex-grow	**flex-shrink**
Actúa cuando	Sobra espacio	Falta espacio
Controla	Crecimiento	Reducción
Valor inicial	`0`	1

flex-basis

Definir el tamaño base del elemento flex

¿Qué hace flex-basis?

`flex-basis` define el tamaño inicial de un elemento sobre el eje principal antes de distribuir el espacio disponible.

Ejemplo

Con `flex-direction: row`, el eje principal es horizontal.
Por lo tanto, `flex-basis` influye inicialmente sobre el ancho.

Ejemplo flex-basis

```
.caja {
    flex-basis: 200px;
}
```

Esto establece una base inicial de 200px sobre el eje principal.

flex-basis con column

Si cambiamos:

```
.contenedor {
    display: flex;
    flex-direction: column;
}
```

entonces el eje principal pasa a ser vertical.

Consecuencia

`flex-basis` se relaciona ahora con el tamaño sobre ese eje vertical.

width frente a flex-basis

En una distribución flex horizontal:

- `width` define un ancho.
- `flex-basis` define la base utilizada por el algoritmo de Flexbox.

Para aprender Flexbox

Cuando queremos controlar el tamaño base de un elemento flex, suele ser útil trabajar directamente con `flex-basis`.

Propiedad flex

Forma abreviada de grow, shrink y basis

¿Qué es la propiedad flex?

`flex` permite escribir en una sola declaración:

- `flex-grow`
- `flex-shrink`
- `flex-basis`

Su estructura general es:

```
flex: grow shrink basis;
```

Ejemplo completo

En lugar de:

```
.caja {
    flex-grow: 1;
    flex-shrink: 1;
    flex-basis: 200px;
}
```

podemos escribir:

```
.caja {
    flex: 1 1 200px;
}
```

Lectura de flex

En:

```
flex: 1 1 200px;
```

tenemos:

- `1`: flex-grow.
- `1`: flex-shrink.
- `200px`: flex-basis.

Una forma muy utilizada

Es frecuente encontrar:

```
.caja {
    flex: 1;
}
```

Esta forma permite crear elementos que participan de manera flexible en el espacio disponible.

Por ahora

Utilizaremos `flex: 1` principalmente para crear elementos que reparten el espacio disponible.

order
Modificar el orden visual

¿Qué hace order?

La propiedad `order` permite modificar el orden visual de un elemento flex.
Por defecto:

```
order: 0;
```

Los elementos con valores menores aparecen antes que aquellos con valores mayores.

Ejemplo

HTML:

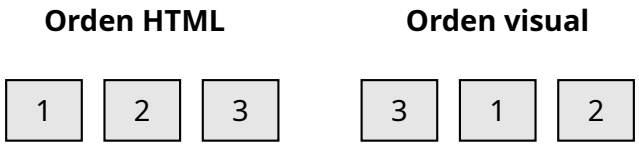
```
<div class="caja uno">1</div>
<div class="caja dos">2</div>
<div class="caja tres">3</div>
```

CSS:

```
.tres {
    order: -1;
}
```

La caja 3 puede aparecer visualmente antes que las demás.

Orden HTML frente a orden visual



Precaución

Cambiar el orden visual no modifica el orden real del documento HTML.

Uso responsable de order

`order` puede ser útil para ajustes visuales, pero no debería utilizarse para corregir una estructura HTML mal organizada.

Buena práctica

El HTML debería conservar un orden lógico y comprensible incluso sin CSS.

align-self

Alinear un elemento de forma individual

¿Qué hace align-self?

`align-self` permite cambiar la alineación de un único elemento sobre el eje transversal.
Puede sobrescribir para ese elemento el comportamiento definido mediante:

`align-items`

Ejemplo

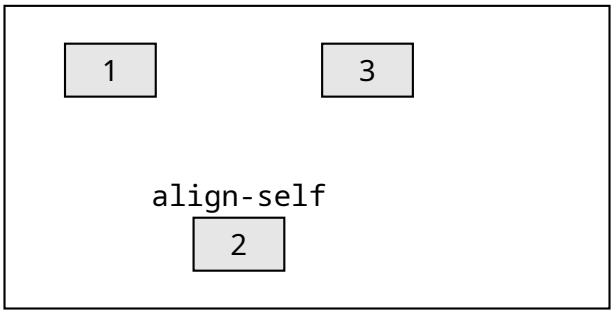
Contenedor:

```
.contenedor {
    display: flex;
    height: 250px;
    align-items: flex-start;
}
```

Un elemento:

```
.destacada {
    align-self: flex-end;
}
```

Representación de align-self



Solo la caja 2 cambia su alineación.

Contenedor y elementos flex

Identificar dónde se escribe cada propiedad

Propiedades del contenedor

Normalmente se escriben en el padre:

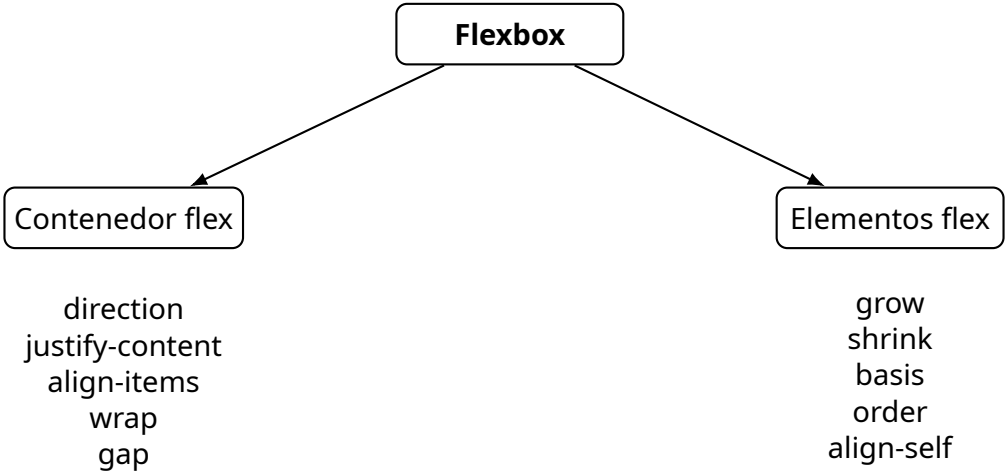
- `display: flex`
- `flex-direction`
- `justify-content`
- `align-items`
- `gap`
- `flex-wrap`
- `align-content`

Propiedades de los elementos

Normalmente se escriben en los hijos:

- `flex-grow`
- `flex-shrink`
- `flex-basis`
- `flex`
- `order`
- `align-self`

Mapa conceptual de Flexbox



Práctica guiada

Sección de servicios

Contexto

Una empresa necesita presentar tres servicios mediante tarjetas.
Construiremos una sección que:

- Organice tarjetas horizontalmente.
- Mantenga separación uniforme.
- Permita varias líneas.
- Distribuya el espacio disponible.
- Mantenga los botones alineados.

Estructura del proyecto

Cree:

```
servicios-flex/
├── index.html
└── css/
└── styles.css
```

Paso 1: HTML

```
<section class="servicios">
    <article class="tarjeta">
        <h2>Diseño Web</h2>
        <p>Diseño de interfaces modernas.</p>
        <button>Consultar</button>
    </article>
    <article class="tarjeta">
        <h2>Soporte</h2>
        <p>Asistencia para plataformas digitales.</p>
        <button>Consultar</button>
    </article>
    <article class="tarjeta">
        <h2>Desarrollo</h2>
        <p>Construcción de aplicaciones web.</p>
        <button>Consultar</button>
    </article>
</section>
```

Paso 2: activar Flexbox

```
.servicios {
    display: flex;
}
```

Observe el cambio en la distribución.

Paso 3: separación y wrap

Agregue:

```
.servicios {
    display: flex;
    gap: 20px;
    flex-wrap: wrap;
}
```

Paso 4: tamaño base

Ahora aplicamos:

```
.tarjeta {
    flex-basis: 250px;
    padding: 20px;
    border: 1px solid gray;
}
```

Observe

Cada tarjeta parte de una base aproximada de 250px.

Paso 5: permitir crecimiento

Modifique:

```
.tarjeta {
    flex-grow: 1;
    flex-basis: 250px;
}
```

Ahora las tarjetas pueden utilizar el espacio disponible.

Paso 6: forma abreviada

Podemos reemplazar:

```
.tarjeta {
    flex-grow: 1;
    flex-shrink: 1;
    flex-basis: 250px;
}
```

por:

```
.tarjeta {
    flex: 1 1 250px;
}
```

Paso 7: Flexbox dentro de la tarjeta

Una caja también puede convertirse en contenedor flex:

```
.tarjeta {
    display: flex;
    flex-direction: column;
}
```

Importante

Un elemento puede ser simultáneamente:
elemento flex de su padre
y contenedor flex de sus propios hijos.

Paso 8: enviar el botón al final

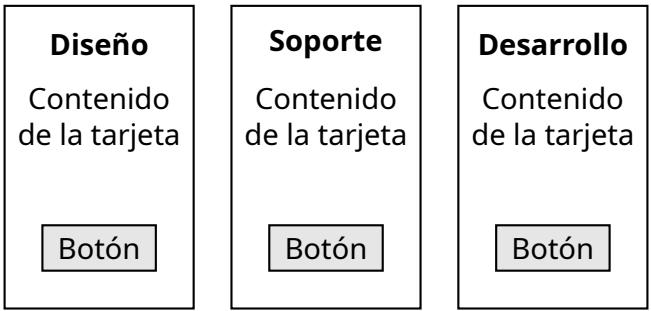
Aplicamos:

```
.tarjeta button {
    margin-top: auto;
}
```

Resultado

El margen automático utiliza el espacio disponible y ayuda a mantener los botones hacia la parte inferior de las tarjetas.

Resultado conceptual



Los botones permanecen visualmente alineados en la parte inferior.

CSS principal de la práctica

```css
.servicios {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}
.tarjeta {
    flex: 1 1 250px;
    display: flex;
    flex-direction: column;
    padding: 20px;
    border: 1px solid gray;
}
.tarjeta button {
    margin-top: auto;
}
```

Ejercicio autónomo

Catálogo de cursos

Situación propuesta

Construya una página para presentar un pequeño catálogo de cursos.
Debe incluir al menos cuatro cursos, por ejemplo:

- Programación.
- Diseño.
- Bases de datos.
- Redes.

Cada curso debe mostrarse dentro de una tarjeta.

Contenido de cada tarjeta

Cada tarjeta debe incluir:

- Nombre del curso.
- Descripción breve.
- Duración.
- Modalidad.
- Botón de información.

Requisitos Flexbox

Utilice como mínimo:

- `display: flex`.
- `flex-direction`.
- `justify-content`.
- `align-items`.
- `gap`.
- `flex-wrap`.
- `flex-basis` o `flex`.

Requisito adicional

Dentro de cada tarjeta:

- Utilice Flexbox en dirección vertical.
- Mantenga el botón hacia la parte inferior.

Objetivo

Comprender que Flexbox puede utilizarse tanto para distribuir tarjetas como para organizar el contenido interno de cada tarjeta.

Estructura de entrega

Organice el proyecto:

```
catalogo-cursos-nombre-apellido/
├── index.html
└── css/
└── styles.css
```

Lista de verificación

Antes de entregar:

- El archivo CSS está correctamente conectado.
- Existe un contenedor flex principal.
- Las tarjetas pueden cambiar de línea.
- Existe separación mediante gap.
- Los elementos se distribuyen correctamente.
- Se utiliza correctamente el eje principal.
- Se utiliza correctamente el eje transversal.
- Las tarjetas tienen un tamaño base definido.
- El código HTML conserva un orden lógico.
- El aprendiz puede explicar cada propiedad utilizada.

Errores frecuentes

Problemas comunes con Flexbox

Aplicar propiedades en el lugar incorrecto

Incorrecto:

```
.caja {
    justify-content: center;
}
```

si `.caja` solamente es un elemento flex.
Para distribuir las cajas:

```
.contenedor {
    display: flex;
    justify-content: center;
}
```

Confundir contenedor e hijo

Recuerde:

$$\text{Padre} \rightarrow \text{organiza}$$

$$\text{Hijo} \rightarrow \text{se adapta}$$

Algunas propiedades pertenecen al contenedor y otras al elemento flex.

Memorizar horizontal y vertical

Otro error es asumir:

justify = horizontal
align = vertical

La interpretación correcta es:

justify $\rightarrow$ eje principal
align $\rightarrow$ eje transversal

y ambos dependen de:

`flex-direction`

Esperar align-content con una sola línea

Si tenemos:

```
.contenedor {
    display: flex;
    align-content: center;
}
```

pero existe solamente una línea, probablemente no veremos el resultado esperado.

Recuerde

`align-content` trabaja con múltiples líneas flex.

Usar order para corregir HTML

Aunque CSS permite modificar el orden visual:

`order`

el HTML debe seguir conservando una secuencia lógica.

Buena práctica

No dependa de `order` para reparar una estructura HTML incorrecta.

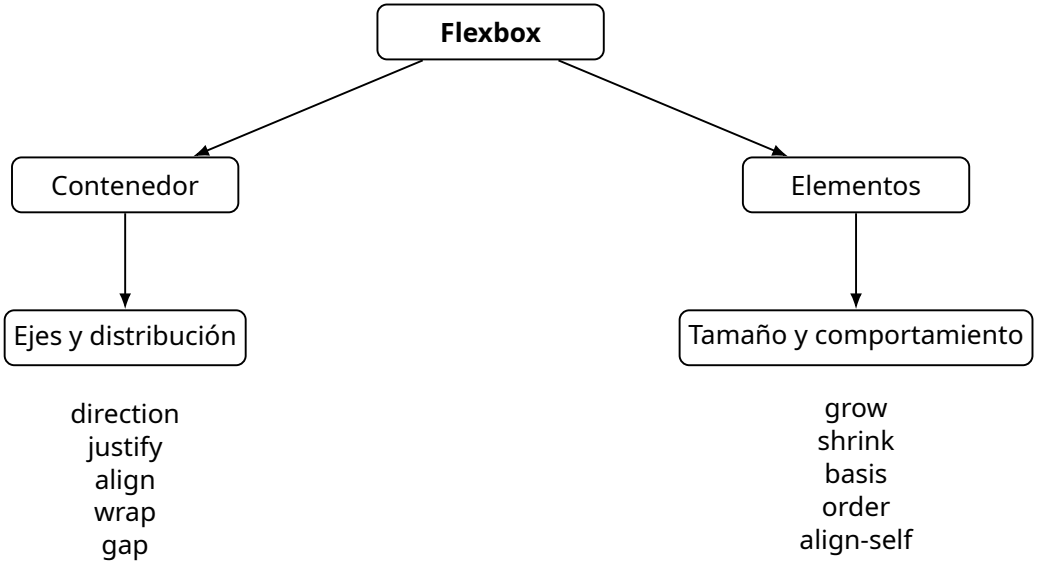
Cierre
Resumen de Flexbox

¿Qué aprendimos?

Flexbox permite:

- Organizar elementos dentro de un contenedor.
- Cambiar la dirección del eje principal.
- Distribuir elementos sobre el eje principal.
- Alinear elementos sobre el eje transversal.
- Crear separaciones mediante gap.
- Permitir múltiples líneas mediante `flex-wrap`.
- Distribuir líneas mediante `align-content`.
- Controlar crecimiento y reducción.
- Definir tamaños base flexibles.
- Modificar individualmente ciertos elementos.

Mapa final de Flexbox



Propiedades esenciales

Propiedad	Función principal
`display: flex`	Activa Flexbox.
`flex-direction`	Define el eje principal.
`justify-content`	Distribuye sobre el eje principal.
`align-items`	Alinea sobre el eje transversal.
`gap`	Separa los elementos.
`flex-wrap`	Permite varias líneas.
`align-content`	Distribuye las líneas.
`flex`	Controla crecimiento, reducción y base.

¿Qué sigue?

Flexbox resulta especialmente útil cuando queremos organizar elementos principalmente en una dimensión.
Sin embargo, algunas interfaces necesitan controlar simultáneamente:

- filas;
- columnas.

Próxima clase

CSS Grid y diseño responsivo
Aprenderemos a construir distribuciones bidimensionales y adaptables.